Chapter 8
Programming

IN THIS CHAPTER YOU WILL:

- learn how to write programs using pseudocode
- use variables and constants
- learn about the appropriate use of basic data types
- write programs that use input and output
- write programs that use sequence
- write programs that use arithmetic operators
- write programs that use selection including IF and CASE statements
- write programs that include logical and Boolean operators
- write programs that use iteration including count-controlled, pre-condition and post-condition loops
- write programs that use totalling and counting
- write programs that perform the string handling methods length and substring
- write programs that use nested statements
- understand the purpose of subroutines
- understand the differences between procedures and functions
- write programs that use subroutines
- understand the purpose and use of parameters
- write programs with subroutines that take parameters

CONTINUED

- write programs with the library routines MOD, DIV, ROUND and RANDOM
- understand what makes a program maintainable
- add features to programs to improve the maintainability
- understand the use of arrays as data structures
- write programs using 1-dimensional arrays
- write programs using 2-dimensional arrays
- understand the need to store data in files
- write programs to read data from a file
- write programs to write data to a file.

GETTING STARTED

Find a computing program (or part of a program) that is written in the programming language you will be using in-lesson. Work with a friend to identify what each of the different lines of code do. Present your findings to the class by showing them each line of code and explaining its purpose.

PROGRAMMING LANGUAGES

There are lots of different procedural programming languages, for example, Java, VB.NET, Python, C++ and many, many more. They all have different commands, requirements for brackets, some need semi-colons at the end of each line of code. There are, however, a set of programming fundamentals that work in the same way in most (there are always one or two that try to be different) languages. Once you know what these fundamentals are, and how these work, then to program in a new language you just need to check the syntax which that specific language uses.

For example, a FOR loop (a type of count-controlled loop that you will learn about later in the chapter) that will output the numbers 1 to 10.

In pseudocode (a generic non-language specific language) this could be:

```
FOR Count ← 1 TO 10
    OUTPUT(Count)
NEXT Count
```

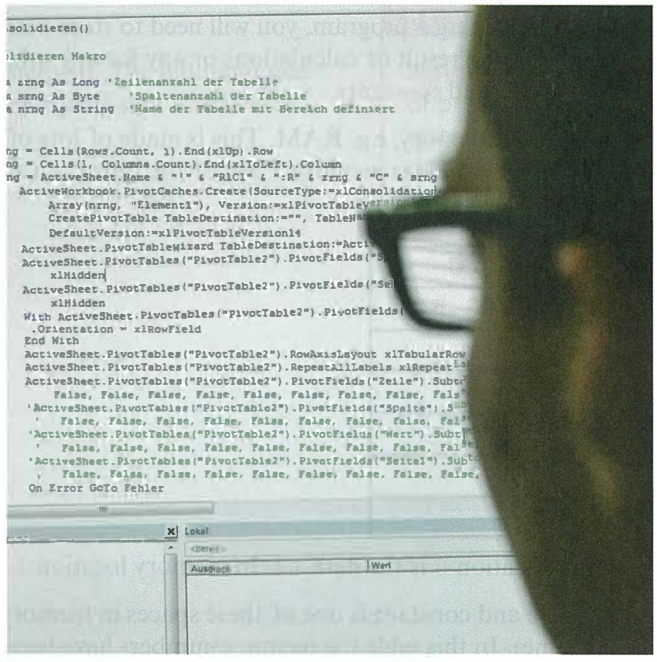


Figure 8.1: Image of a man reading program code

CONTINUED

In VB.NET:

```
FOR count = 1 to 10
    Console.writeline(count)
NEXT count
```

In Java:

```
for(count = 1; count < 11; count++){
    System.out.println(count);
}
```

These all do the same functions, and they all follow the same principles. They start with the word 'for'. They have a variable (count). They set the starting value and they say when to stop. So once you can do a for loop in VB.NET, you should just be able to search for how to do it in Python and just change a couple of the words.

Discussion questions

1 Why do you think all these languages have similar constructs?

2 Find some constructs that are in one language, e.g. VB.NET, and not in another, e.g. Python.

8.1 Programming concepts

This chapter will introduce different code examples. It will show how to do each of the procedures in three different languages (Java, VB.NET and Python) and pseudocode. The pseudocode (red font) will always appear as the first code in each example.

Variables and constants

What are variables and constants?

When you are writing a program, you will need to store data; whether this is data that has been input, the result of calculations or any for any other reason. You can store data in **variables** and **constants**.

A computer has memory, e.g. RAM. This is made of lots of spaces where you can put data. Imagine this table is memory. Each of the boxes can store a piece of data.

0	10
1	Red
2	22.5
3	True
4	
5	
6	
7	

In memory location `0` is the data `10`. In memory location `1` is the data `red`.

Each variable and constant is one of these spaces in memory that is given an **identifier** (it has a name). In this table the memory numbers have been replaced with their identifiers. The memory space with the name `number1` is storing the piece of data `10`.

KEY WORDS

variable: a named memory location that can store data. The data can change whilst a program is running.

constant: a named memory location that can store data. The data cannot change whilst a program is running.

identifier: a name given to a variable, constant, data structure (e.g. array) or subroutine.

number1	10
colour	red
price	22.5
finished	True
4	
5	
6	
7	

Variables and constants have one difference. In a variable, you can change the data while the program is running. For example, putting the number 30 into memory location number1, the memory would then look like this:

number1	30
colour	red
price	22.5
finished	True
4	
5	
6	
7	

A constant cannot have its value changed while the program is running. When you declare a constant you put a value into it and this cannot be changed.

Using variables

Putting data into a variable is done using an **assignment** statement. The left hand side of the equals sign is the identifier. The right hand side of the equals sign is the value (see Figure 8.2).

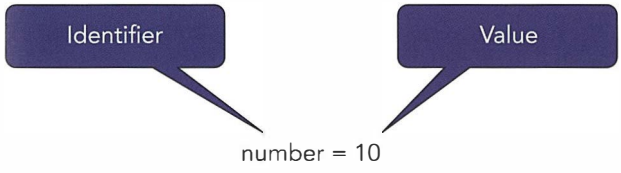


Figure 8.2: Components of an assignment

KEY WORD

assignment: a type of programming statement that stores data in a variable or constant.

```
Number ← 10
Colour ← "red"
Price ← 22.2
```

VB.NET

```
Dim number As Integer
Dim colour As String
Dim price As Single
number = 10
colour = "red"
price = 22.2
```

Python

```
number = 10
colour = "red"
price = 22.2
```

Java

```
public static void main(String args[]){
    Integer number = 10;
    String colour = "red";
    Double price = 22.2;
}
```

To get data out of a variable you just use its identifier (see Figure 8.3).

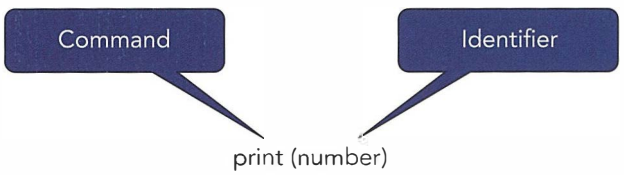


Figure 8.3: Printing the contents of a variable

```
Number ← 10
Colour ← "red"
OUTPUT(Number)
OUTPUT("The colour is ", Colour)
Price ← Number * 2
```

VB.NET

```
Dim number As Integer
Dim colour As String = "red"
Dim price As Single
Console.WriteLine(number)
Console.WriteLine("The colour is " & colour)
price = number * 2
```

Python

```
colour = "red"
number = input()
print("The colour is", colour)
price = number * 2
```

Java

```
public static void main(String args[]){
    Integer number = 10;
    String colour = "red";
    System.out.println("The colour is " + colour);
    Integer price = number * 2;
}
```

Using constants

Before you use a constant in your program you need to give it a value. This is an assignment statement the same as a variable. (No examples are given for Python as it does not have in-built constants.)

```
CONSTANT Colour ← "yellow"
```

VB.NET

```
Const colour As String = "yellow"
```

Java

```
public static void main(String args[]){
    final String colour = "yellow";
}
```

Using the key word constant makes it clear that this value cannot then be changed.

You get data out of a constant the same way as a variable, by using its identifier.

```
OUTPUT(Colour)
```

VB.NET

```
Const colour As String = "yellow"
Console.Writeline(colour)
```

Java

```
public static void main(String args[]){
    final String colour = "yellow";
    System.out.println(colour);
}
```

8.2 Data types

Data in programs can be of different types. For example, it could be numeric or text. You will need to tell your program what type of data you want it to store. Some programming languages need you to declare what type of data your variable will store when you first use it. In some programming languages you need to swap data between types, for example, Python will only output **string** (text) data, so if you try and output a number without turning it into a string it will produce an error.

Table 8.1 shows common **data types**:

Data type	Description	Example data
String	Text – characters numbers and symbols. The data will always need to be inside speech marks, either ' ' or " ".	"hello" "123" "help!"
Integer	Whole numbers.	1 23 -300 45656
Real, **single**, **double**	Decimal numbers.	1.2 23.0 -20.49 3949.3834
Boolean	Either true or false.	TRUE FALSE
Char	One character or number of symbol. The data will always need to be inside speech marks, either '' or "".	"h" "9" "?"

Table 8.1: Common data types

KEY WORDS

string: a data type. Any characters including letters, numbers and/or symbols.

data type: the characteristics of a piece of data. Common data types are string, integer, real and Boolean.

integer: a data type. Whole numbers.

real: a data type. Decimal numbers.

single: a data type. Decimal numbers.

double: a data type. Decimal numbers.

Boolean: a data type. True or False.

char: A single character, e.g. 'A', '1', '*'.

ACTIVITY 8.1

Take each data type in turn and think of at least 10 different examples of data that can be stored (apart from Boolean where there can be only two). From these, identify whether any of these could be more than one data type, discuss in pairs what options would be valid and which would be most appropriate.

Peer Assessment

Compare your work with another pair. Identify if all of the data is appropriate for the data type chosen. Discuss the choices, e.g. if one was more appropriate than the other?

Storing different data types

Storing a string in a variable:

```
Colour ← "red"
```

Storing an integer in a constant:

```
CONSTANT Value ← 10
```

Storing a real number in a variable:

```
Price ← 22.4
```

Storing a Boolean in a variable:

```
Flag ← TRUE
```

VB.NET

```
Dim colour As String = "red"
Const value As Integer = 10
Dim price As Single = 22.4
Dim flag As Boolean = True
```

Python

```
colour = "red"
value = 10
price = 22.4
flag = True
```

Java

```
public static void main(String args[]){
    String colour = "red";
    final Integer value = 10;
    Double price = 22.4;
    Boolean flag = true;
}
```

Converting between data types

You might need to turn one data type into another data type. This is not required as part of the specification, but when you are programming in your chosen language you might have to do it for your program to work. This is called **casting**. You can do this by using the name of the data type you want the data to become.

KEY WORD

casting: converting data from one data type to another data type.

Example 1

Convert a string to an integer:

```
Number ← int("123")
```

VB.NET

```
Dim number As Integer
number = Convert.ToInt16("123")
```

Python

```
number = int("123")
```

Java

```
public static void main(String args[]){
    Integer number = Integer.parseInt("123");
}
```

Example 2

Convert a number to a string:

```
Value ← string(22.4)
```

VB.NET

```
Dim value As String
value = Convert.ToString(22.4)
```

Python

```
value = str(22.4)
```

Java

```
public static void main(String args[]){
    String value = Double.toString(22.4);
}
```

Questions

1 Tick one or more boxes in each row to identify whether each statement refers to variables and/or constants.

Statement	Variable	Constant
You cannot change the value when the program is running.		
It has an identifier.		
It is a memory location.		
You can change its value when the program is running.		
It stores a piece of data.		

2 Write a pseudocode statement to assign the word `"house"` to a variable named `MyWord`.

3 Write a pseudocode statement to declare a constant named `MultiplyValue` with the value `10`.

8.3 Input and output

Output

A program at some point will probably need to give information to the user. It does this using **output**. When outputting strings (characters, letters, etc.).

KEY WORD

output: data that is displayed to the user usually on-screen.

Example 1

Output the words, Hello World:

```
OUTPUT("Hello world")
```

VB.NET

```
Console.WriteLine("Hello World")
```

Python

```
print("Hello World!")
```

Java

```
public static void main(String args[]){
    System.out.println("Hello World");
}
```

Example 2

Output the number 20:

```
OUTPUT(20)
```

VB.NET

```
Console.WriteLine(20)
```

Python

```
print(20)
```

Java

```
public static void main(String args[]){
    System.out.println(20);
}
```

If you want to output more than one piece of data then you can join them using a **concatenation** symbol. Concatenation means join together, so it joins multiple pieces of data together. This could be a comma (,), an ampersand (&) or a plus (+) depending on your language. All are acceptable in pseudocode.

KEY WORD

concatenation: joining two or more strings together.

Example 1

Output the word Hello, then the contents of variable name:

```
OUTPUT("Hello ", Name)
```

VB.NET

```
Dim name As String = "Alex"
Console.WriteLine("Hello " & Name)
```

Python

```
name = "Alex"
print("Hello", name)
```

Java

```
public static void main(String args[]){
    String name = "Alex";
    System.out.println("Hello " + name);
}
```

Example 2

Output the cost of an item stored in the variable cost:

```
OUTPUT("The cost is " , Cost)
```

VB.NET

```
Dim cost As Single = 22.54
Console.WriteLine("The cost is " & cost)
```

Python

```
cost = 22.54
print("The cost is", cost)
```

Java

```
public static void main(String args[]){
    Double cost = 22.54;
    System.out.println("The cost is " + cost);
}
```

Example 3

Output the number of balloons stored in the variable balloon:

```
OUTPUT("There are " , Balloon , " balloons")
```

VB.NET

```
Dim balloon As Integer = 100
Console.WriteLine("There are " & balloon & "balloons")
```

Python

```
balloon = 100
print("There are", balloon, "balloons")
```

Java

```
public static void main(String args[]){
    Integer balloon = 100;
    System.out.println("There are " + balloon + " balloons");
}
```

In these examples you will see there are spaces within the speech marks. This is because `OUTPUT ("Hello", Name)` would join these together, e.g. `HelloJane`. When writing in pseudocode it is not important that these are included, but you might need to do it when outputting in your chosen programming language.

Input

A program might need the user to enter (**input**) some data. To do this, the command word `INPUT` is used. This cannot appear on its own, otherwise the data entered will disappear into space. So you need to do something with it, for example, store it in a variable.

KEY WORD

input: the user entering data into the program, usually from a keyboard.

Example 1

Input a number and store it in a variable:

```
INPUT Number
```

VB.NET

```
Dim number As Integer
number = Console.ReadLine
```

Python

```
number = int(input())
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    Integer number = Integer.parseInt(scanner.nextLine());
}
```

Example 2

Tell the user to enter a word and store it in a variable:

```
OUTPUT("Enter a word")
INPUT Word
```

VB.NET

```
Dim word As String
Console.WriteLine("Enter a word")
word = Console.ReadLine
```

Python

```
number = input("Enter a word")
```

Java

```
public static void main(String args[]){
    System.out;println("Enter a word")
    Scanner scanner = new Scanner(System.in);
    String word = scanner.nextLine();
}
```

PROGRAMMING TASK 8.1

A program asks the user to register for a new account. The user needs to enter key information, e.g. name, date of birth, select a username, etc.

Getting started

1. Work in pairs to list the different items that the program will collect.
2. Identify the most appropriate data type for each of the items you have identified.

CONTINUED

Practice

1. Select appropriate variables for the items you have identified that you are going to store.
2. Write a program to ask the user to enter each of the items in turn. Read in each value and store it in an appropriate variable.
3. Output a message confirming the details that the user has entered.

Challenge

1. The username needs to be at least 8 characters long. Find out how to work out the length of a string input and output how many characters the user has entered.
2. Find out how to use selection statements to check the length of the string and if it is not long enough, ask the user to enter a different username.

8.4 Arithmetic operators

Arithmetic operators instruct a program to perform calculations. Table 8.2 describes the most common operators, many of which you will know from mathematics.

KEY WORD

arithmetic operator: a symbol that performs a mathematical function, e.g. '+' adds two values together.

Operator	Description	Example
+	Adds two values together.	`10 + 2` gives 12 `11.3 + 9` gives 20.3
-	Subtracts the second value from the first.	`10 - 2` gives 8 `11.3 - 9` gives 2.3
*	Multiplies two values together.	`10 * 2` gives 20 `11.3 * 9` gives 101.7
/	Divides the first number by the second.	`10 / 2` gives 5 `11.3 / 9` gives 1.256
DIV	Gives the whole number after the first number is divided by the second, i.e. it ignores any decimals.	`DIV(10, 2)` gives 5 `DIV(11, 9)` gives 1
MOD	Gives the remainder after the first number is divided by the second, i.e. how many are left.	`MOD(10, 2)` gives 0 `MOD(11, 9)` gives 2
^	Power of.	`2 ^ 3 = 8` `3 ^ 2 = 9`

Table 8.2: Common operators

MOD has one special use in programming. It can be used to work out if a number is odd or even. If you perform MOD 2 to a number and it returns 0 then it is even, if it returns 1 then it is odd.

Example: MOD(10, 2) = 0 therefore 10 is even.

MOD(11, 2) = 1 therefore 11 is odd.

SKILLS FOCUS 8.1

MOD VS DIV

It is important that you know the difference between MOD and DIV. They are similar in their function, but are often confused with division (/). You need to be able to use both of these, both to identify the result of the operation, and to be able to write programs using them. In this Skills Focus you will be calculating the result from a MOD and DIV operation.

DIV gives the whole number after the division and ignores any remainder.

a 10/2 =v5 There is no remainder, so DIV(10, 2) = 5.

b 20/7 = 2.857 There is a remainder, so DIV(20, 7) = 2 (ignore the numbers after the decimal point).

c 100/21 = 4.762 There is a remainder, so DIV(100, 21) = 4 (ignore the numbers after the decimal point).

MOD gives the remainder after division. This is not the decimal point, but how many values are left.

a 10/2 = 5 There is no remainder, so MOD(10, 2) = 0.

b 20/7 = 2.857 There is a remainder. Take the DIV result (2) and multiply it by the divisor number. 7 * 2 = 14. The remainder is how many more numbers are between 14 and the 20 (20 − 6). The answer is 6.

c 100/21 = 4.762 There is a remainder. Take the DIV result (4) and multiply it by the divisor 21 * 4 = 84. The remainder is 100 − 84 which is 16.

d 30/9 = 3.3333 There is a remainder. 9 * 3 = 27. 30 − 27 = 3.

Questions

1 Calculate the result for each of these equations:
   a DIV(9, 2) **b** DIV(17, 3)

2 Calculate the result for each of these equations:
   a MOD(9, 2) **b** MOD(17, 3)

KEY WORDS

MOD: remainder division. The remainder after the division is performed, e.g. `MOD(5, 2) = 2`.

DIV: integer division. The remainder from the division is ignored, e.g. `DIV(5, 2) = 2`.

COMPUTER SCIENCE IN CONTEXT

Many of these arithmetic operators should be familiar to you from mathematics, where you should be used to working out expressions. In programming the same principles are used, you write the formulae but not the answer - the computer works that out because the input data can be changed.

The operators for DIV and MOD will differ depending on the programming language you are using.

Example 1

Taking two numbers as input and adding them together:

```
OUTPUT("Enter the first number")
INPUT Num1
OUTPUT("Enter the second number")
INPUT Num2
Total ← Num1 + Num2
```

VB.NET

```
Dim num1 As Integer
Dim num2 As Integer
Dim total As Integer
Console.WriteLine("Enter the first number")
num1 = Console.ReadLine
Console.WriteLine("Enter the second number")
num2 = Console.ReadLine
total = num1 + num2
```

Python

```
num1 = int(input("Enter the first number"))
num2 = int(input("Enter the second number"))
total = num1 + num2
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter the first number");
    Integer num1 = Integer.parseInt(scanner.nextLine());
    System.out.println("Enter the second number");
    Integer num2 = Integer.parseInt(scanner.nextLine());
    Integer total = num1 + num2;
}
```

Example 2

Subtracting 10 from 20:

```
Number1 ← 10
Number2 ← 20
Result ← Number2 - Number1
```

VB.NET

```
Dim number1 As Integer = 10
Dim number2 As Integer = 20
Dim result As Integer
result = number2 - number1
```

Python

```
number1 = 10
number2 = 20
result = number2 - number1
```

Java

```
public static void main(String args[]){
    Integer number1 = 10;
    Integer number2 = 20;
    Integer result = number2 - number1;
}
```

Example 3

Multiplying two values together:

```
Number1 ← 5
OUTPUT("Enter a number")
INPUT Number2
Result ← Number1 * Number2
```

VB.NET

```
Dim number1 As Integer = 5
Dim number2 As Integer
Console.WriteLine("Enter a number")
number2 = Console.ReadLine
Dim result As Integer
result = number2 * number1
```

Python

```
number1 = 5
number2 = int(input("Enter a number"))
result = number1 * number2
```

Java

```
public static void main(String args[]){
    Integer number1 = 5;
    System.out.println("Enter a number");
    Scanner scanner = new Scanner(System.in);
    Integer number2 = Integer.parseInt(scanner.nextLine());
    Integer result = number1 * number2;
}
```

Example 4

Dividing 100 by 5:

```
Number1 ← 100
Number2 ← 5
Result ← Number1 / Number2
```

VB.NET

```
Dim number1 As Integer = 100
Dim number2 As Integer = 5
Dim result As Single
result = number2 / number1
```

Python

```
number1 = 100
number2 = 5
result = number2 / number1
```

Java

```
public static void main(String args[]){
    Double number1 = 100.0;
    Double number2 = 5.0;
    Double result = number2 / number1;
}
```

Example 5

Finding the whole number after dividing 33 by 7:

```
Result ← DIV(33, 7)
```

VB.NET

```
Dim result As Single
result = 33 \ 7
```

Python

```
result = int(33 / 7)
```

Java

```
public static void main(String args[]){
    Integer result = 33 / 7;
}
```

Example 6

Finding the remainder after dividing 33 by 7:

```
Result ← MOD(33, 7)
```

VB.NET

```
Dim result As Single
result = 33 Mod 7
```

Python

```
result = 33 % 7
```

Java

```
public static void main(String args[]){
    Integer result = 33 % 7;
}
```

Calculations can use **parentheses** (brackets) to change the order the calculations are performed in. The calculations within the brackets are done first.

KEY WORD

parentheses: brackets in a mathematical statement. They determine which calculations are performed first.

Example 7

```
Total ← 1 + (2 * 3)
Total ← (1 + 2) * 3
```

The first line will result in 7 (3 * 2 = 6 + 1 = 7).

The second line will result in 9 (1 + 2 = 3 * 3 = 9).

8.5 Sequence

Sequence is the first of three constructs within programs. A sequence is a series of statements that are executed (run) once, in the order they are written.

KEY WORD

sequence: a programming construct. Instructions are run once and in the order they are written.

Example 1

```
OUTPUT("Enter a colour")
INPUT Colour
OUTPUT("Enter your name")
INPUT Name
OUTPUT(Name , " your favourite colour is " , Colour)
```

VB.NET

```
Dim colour As String
Console.WriteLine("Enter a colour")
colour = Console.ReadLine()
Dim name As String
Console.WriteLine("Enter your name")
```

```
name = Console.ReadLine()
Console.WriteLine(name & " your favourite colour is " & colour)
```

Python

```
colour = input("Enter a colour")
name = input("Enter your name")
print(name, "your favourite colour is", colour)
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a colour");
    String colour = scanner.nextLine();
    System.out.println("Enter your name");
    String name = scanner.nextLine();
    System.out.println(name + " your favourite colour is " + colour);
}
```

This is a sequence. It has 3 lines are executed once, and in the order they are written (line 1, then 2 then 3).

Example 2

```
X ← 1
Y ← 2
Z ← 3
Total ← X + Y + Z
OUTPUT("Enter the first value")
INPUT Value1
```

VB.NET

```
Dim X As Integer = 1
Dim Y As Integer = 2
Dim Z As Integer = 3
Dim total As Integer = X + Y + Z
Dim value1 As String
Console.WriteLine("Enter the first value")
value1 = Console.ReadLine
```

Python

```
x = 1
y = 2
z = 3
total = x + y + z
value1 = int(input("Enter the first value"))
```

Java

```
public static void main(String args[]){
    Integer X = 1;
    Integer Y = 2;
    Integer Z = 3;
    Integer total = X + Y + Z;
    String value1;
    System.out.println("Enter the first value");
    Scanner scanner = new Scanner(System.in);
    value1 = scanner.nextLine();
}
```

Questions

4 Give the result from the following calculations:

a `10 + 20`
b `20 / 2`
c `100 * 4`
d `50 - 15`
e `20 DIV 2`
f `39 DIV 6`
g `20 MOD 2`
h `40 MOD 6`

5 Write a program to take a number as input, multiply it by 2 and output the result.

6 Write a program to store the numbers 10 and 12 in constants, add them together and then output the result.

7 Write a program to ask a user to enter their age and name, then output a message that uses both values, e.g. Hello Suzie you are 15 year old.

KEY WORDS

selection: a programming construct. A condition is checked and this determines which code is run, or not run.

IF statement: a type of selection construct where the result of the condition is either true or false.

CASE statement: a type of selection construct where there is a list of different values to compare a single value against.

logical operator: a symbol that performs a comparison resulting in True or False. Can be equals, not equal to, less than, less than or equal to, greater than, greater than or equal to.

8.6 Selection

Selection is the second of the three constructs. In selection a condition is checked and this determines which, if any, code is run. There are two forms of selection, **IF statements** and **CASE statements**.

Conditions need **logical operators**. These allow for comparisons to be made. Table 8.3 describes these different operators. Each statement using a logical operator results in TRUE or FALSE.

Logical operator	Description	Example
= or ==	Equals to	`10 = 10?` would give `TRUE`. 10 is equal to 10. `10 = 2?` would give `FALSE`. 10 is not equal to 2.
<> or !=	Not equal to	`10 <> 10?` would give `FALSE`. 10 is not, not equal to 10. `10 <> 2?` would give `TRUE`. 10 is not equal to 2.
<	Less than	`10 < 11?` would give `TRUE`. 10 is less than 11. `10 < 10?` would give `FALSE`. 10 is not less than 10. `11 < 10?` would give `FALSE`. 11 is not less than 10.

(continued)

Logical operator	Description	Example
<=	Less than or equal to	`10 <= 11?` would give `TRUE`. 10 is less than or equal to 10. `10 <= 10?` would give `TRUE`. 10 is less than or equal to 10. `11 <= 10?` would give `FALSE`. 11 is not less than or equal to 10.
>	Greater than	`10 > 11?` would give `FALSE`. 10 is not greater than 11. `10 > 10?` would give `FALSE`. 10 is not greater than 10. `11 > 10?` would give `TRUE`. 11 is greater than 10.
>=	Greater than or equal to	`10 >= 11?` would give `FALSE`. 10 is not greater than or equal to 11. `10 >= 10?` would give `TRUE`. 10 is greater than or equal to 10. `11 >= 10?` would give `TRUE`. 11 is greater than or equal to 10.

Table 8.3: Logical operators

SKILLS FOCUS 8.2

COMPARISON OPERATORS

Comparison operators are used in comparison statements; both selection and iteration. The operators are very similar and you need to know the difference to make sure you know, **a**, how to read the statements to make sure you follow an algorithm correctly, and **b**, which to select when you are writing your own comparison statements.

A common error is when less than and greater than are confused. The shape of them can help you to work out which is correct.

`IF(10 < 2)` The smaller part of the < is nearest the left, the 10. This is the less than part. So the statement reads if 10 is less than 2. This would result in False because 10 is not less than 2.

`IF(150 > 25)` The larger part of the > is nearest the left, the 150. This is the greater than part. So the statement reads if 150 is greater than 25. This would result in True because 150 is greater than 25.

`IF(33 <= 34)` The smaller part of the <= is nearest the left, the 33. This is the less than part. There is also an equals after the less than sign. So the statement reads if 33 is less than or equal to 34. This would result in True, 33 is less than 34.

`IF(50 >= 70)` The larger part of the >= is nearest the left, the 50. This is the greater than part. There is also an equals after the less than sign. So the statement reads if 50 is greater than or equal to 70. This would result in False, 50 is not greater than or equal to 70.

CONTINUED

Questions

1 Put each statement into words:

a `IF(1 < 2)`

Look at the left of the symbol. Is it small or large? Write the first number, followed by the symbol name, then the second number.

b `IF(6 > 3)`

c `IF(999 >= 998)`

d `IF(34 <= 77)`

2 Work out if each statement is True or False.

a `IF(66 < 40)`

b `IF(100 > 101)`

c `IF(90 <= 45)`

d `IF(30 >= 30)`

IF statements

The command `IF` is followed by a condition that is created using the logical operators. There are three stages of IF statements; `IF`, `ELSE` and `ELSEIF`.

`IF` has one comparison and the code inside the `IF` will only run if that condition is True. If it is not true, the code in the `IF` statement will not run.

It follows the structure:

```
IF comparison THEN
    Statements that run if the comparison is true
ENDIF
```

Example 1

This program will check the value in the variable `num1` is equal to 10. If it is, it will output the word True.

```
Num1 ← 10
IF Num1 = 10 THEN
    OUTPUT("True")
ENDIF
```

VB.NET

```
Dim num1 As Integer = 10
If num1 = 10 Then
    Console.WriteLine("True")
End If
```

Python

```
num1 = 10
if num1 == 10:
    print("True")
```

Java

```
public static void main(String args[]){
    Integer num1 = 10;
    if(num1 == 10){
        System.out.println("True");
    }
}
```

Example 2

This program will check if the value input is greater than the one stored in the variable.

```
OUTPUT("Enter a number")
INPUT ValueInput
StoredValue ← 100
IF ValueInput > StoredValue THEN
    OUTPUT("It is more than 100")
ENDIF
```

VB.NET

```
Dim valueInput As Integer
Console.WriteLine("Enter a number")
valueInput = Console.ReadLine
Dim storedValue As Integer = 100
If valueInput > storedValue Then
    Console.WriteLine("It is more than 100")
End If
```

Python

```
valueInput = int(input("Enter a number"))
storedValue = 100
if valueInput > storedValue:
    print("It is more than 100")
```

Java

```
public static void main(String args[]){
    Integer storedValue = 100;
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a number");
    Integer valueInput = Integer.parseInt(scanner.
nextLine());
    if(valueInput > storedValue){
        System.out.println("It is more than 100");
    }
}
```

ELSE

This is added within an `IF` statement. If the `IF` statement's condition is false then the `ELSE` will run. You can only ever have one `ELSE` in an `IF` statement.

`ELSE` follows the structure:

```
IF comparison THEN
    Statements that run if the comparison is true
ELSE
    Statements that run if the comparison is false
ENDIF
```

Example 1

In this example if the two values are the same it outputs `"That's correct"`. If they are not the same then the `ELSE` runs, it will output `"Incorrect"`.

```
Num ← 10
OUTPUT("Enter a number")
INPUT Guess
IF Num = Guess THEN
    OUTPUT("That's correct")
ELSE
    OUTPUT("Incorrect")
ENDIF
```

VB.NET

```
Dim num As Integer = 10
Dim guess As Integer
Console.WriteLine("Enter a number")
guess = Console.ReadLine
If num = guess Then
    Console.WriteLine("That's correct")
Else
    Console.WriteLine("Incorrect")
End If
```

Python

```
num = 10
guess = int(input("Enter a number"))
if num == guess:
    print("That's correct")
else:
    print("Incorrect")
```

Java

```
public static void main(String args[]){
    Integer num = 10;
    Integer guess;
    Scanner scanner = new Scanner(System.in);
```

```
        System.out.println("Enter a number");
        guess = Integer.parseInt(scanner.nextLine());
        if(num == guess){
            System.out.println("That's correct");
        }else{
            System.out.println("Incorrect");
        }
    }
```

Example 2

In this example, it will output the smallest number, or one of the numbers if they are both the same.

```
Value1 ← 10
Value2 ← 20
IF Value1 < Value2 THEN
    OUTPUT(Value1)
ELSE
    OUTPUT(Value2)
ENDIF
```

VB.NET

```
Dim value1 As Integer = 10
Dim value2 As Integer = 20
If value1 < value2 Then
    Console.WriteLine(value1)
Else
    Console.WriteLine(value2)
End If
```

Python

```
value1 = 10
value2 = 20
if value1 < value2:
    print(value1)
else:
    print(value2)
```

Java

```
public static void main(String args[]){
    Integer value1 = 10;
    Integer value2 = 20;
    if(value1 < value2){
        System.out.println(value1);
    }else{
        System.out.println(value2);
    }
}
```

ELSEIF

This allows for a second condition to be used within the same `IF` statement.
If the first condition is False, then a second condition can be checked.

`ELSEIF` follows the structure:

```
IF comparison1 THEN
    Statements that run if the comparison is true
ELSEIF comparison2 THEN
    Statements that run if comparison1 is false, and comparison2 is true
ENDIF
```

Example 1

This will output which number is greater, or nothing will output if they are the same.

```
Num1 ← 10
Num2 ← 20
IF Num1 < Num2 THEN
    OUTPUT(Num2)
ELSEIF Num2 < Num1 THEN
    OUTPUT(Num1)
ENDIF
```

VB.NET

```
Dim num1 As Integer = 10
Dim num2 As Integer = 20
If num1 < num2 Then
    Console.WriteLine(num2)
ElseIf num2 < num1 Then
    Console.WriteLine(num1)
End If
```

Python

```
num1 = 10
num2 = 20
if num1 < num2:
    print(num2)
elif num2 < num1:
    print(num1)
```

Java

```
public static void main(String args[]){
    Integer num1 = 10;
    Integer num2 = 20;
    if(num1 < num2){
        System.out.println(num2);
    }else if(num2 < num1){
        System.out.println(num2);
    }
}
```

You can use multiple ELSEIF statements, and combine them with a single ELSE statement at the end.

This will follow the structure:

```
IF comparison1 THEN
    Statements that run if the comparison is true
ELSEIF comparison2 THEN
    Statements that run if comparison1 is false, and comparison2 is true
….as many ELSEIFs as you need
ELSE
    Statements that run if none of the comparisons are true
ENDIF
```

Example 2

This uses ELSEIF and an ELSE to output the largest number.

```
IF Num1 > Num2 THEN
    OUTPUT(Num1)
ELSEIF Num2 > Num1 THEN
    OUTPUT(Num2)
ELSE
    OUTPUT("They are the same")
ENDIF
```

VB.NET

```
Dim num1 As Integer = 10
Dim num2 As Integer = 20
If num1 > num2 Then
    Console.WriteLine(num1)
ElseIf num2 > num1 Then
    Console.WriteLine(num2)
Else
    Console.WriteLine("They are the same")
End If
```

Python

```
num1 = 10
num2 = 20
if num1 > num2:
    print(num1)
elif num2 > num1:
    print(num2)
else:
    print("They are the same")
```

Java

```
public static void main(String args[]){
    Integer num1 = 10;
    Integer num2 = 20;
```

```
    if(num1 < num2){
        System.out.println(num2);
    }else if(num2 < num1){
        System.out.println(num2);
    }else{
        System.out.println("They are the same");
    }
}
```

Example 3

This uses multiple ELSEIFs.

```
IF Age < 14 THEN
    OUTPUT("You are not old enough")
ELSEIF Age < 16 THEN
    OUTPUT("You need an adult present")
ELSEIF
```

VB.NET

```
Dim age As Integer
Console.WriteLine("Enter your age")
age = Console.ReadLine
If age < 14 Then
    Console.WriteLine("You are not old enough")
ElseIf age < 16 Then
    Console.WriteLine("You need an adult present")
End If
```

Python

```
age = int(input("Enter your age"))
if age < 14:
    print("You are not old enough")
elif age < 16:
    print("You need an adult present")
```

SELECT CASE

A SELECT CASE statement allows the program to take one variable or value, and then have lots of different options depending what it is equal to.

CASE follows the structure:

```
CASE OF variable
    value1:
        Statements that run if CASE value1 is true
    value2:
        Statements that run if CASE value1 is false, and value2 is true
    OTHERWISE
        Statements that run if none of the comparisons are true.
ENDCASE
```

A case can have as many CASE statements as needed, but can only ever have a maximum of one default (this runs if none of the comparisons are true).

Example 1

Using a SELECT CASE to output a grade for an in-lesson test. (No example is given for Python as it does not have a CASE construct and no example is given for Java as it does not support switch statements with comparisons, e.g. < or >.)

```
Score ← INPUT("Enter score")
CASE OF score:
    >=80: OUTPUT ("A")
    >=70: OUTPUT("B")
    >=60: OUTPUT("C")
    >=50: OUTPUT("D")
    OTHERWISE OUTPUT("U")
ENDCASE
```

VB.NET

```
Dim score As Integer
Console.WriteLine("Enter score")
score = Console.ReadLine
Select Case score
    Case >= 80
        Console.WriteLine("A")
    Case >= 70
        Console.WriteLine("B")
    Case >= 60
        Console.WriteLine("C")
    Case >= 50
        Console.WriteLine("D")
    Case Else
        Console.WriteLine("U")
End Select
```

Example 2

Output a message depending on which number is entered. (No example is given for Python as it does not have a CASE construct.)

```
OUTPUT("Enter a number, 1 to 5")
INPUT Choice
CASE OF Choice:
    1: OUTPUT("Menu option 1")
    2: OUTPUT("Menu option 2")
    3: OUTPUT("Menu option 3")
    4: OUTPUT("Menu option 4")
    5: OUTPUT("Menu option 5")
    OTHERWISE OUTPUT("Invalid choice")
ENDCASE
```

VB.NET

```
Dim choice As Integer
Console.WriteLine("Enter a number, 1 to 5")
choice = Console.ReadLine
Select Case choice
    Case 1
        Console.WriteLine("Menu option 1")
    Case 2
        Console.WriteLine("Menu option 2")
    Case 3
        Console.WriteLine("Menu option 3")
    Case 4
        Console.WriteLine("Menu option 4")
    Case 5
        Console.WriteLine("Menu option 5")
    Case Else
        Console.WriteLine("Invalid choice")
End Select
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a number, 1 to 5");
    Integer choice= Integer.parseInt(scanner.nextLine());
    switch(choice){
        case 1:
            System.out.println("Menu option 1");
            break;
        case 2:
            System.out.println("Menu option 2");
            break;
        case 3:
            System.out.println("Menu option 3");
            break;
        case 4:
            System.out.println("Menu option 4");
            break;
        case 5:
            System.out.println("Menu option 5");
        default:
            System.out.println("Invalid choice");
    }
}
```

ACTIVITY 8.2

What is the difference between IF and CASE statements? Is there a scenario when one is more appropriate than another? Write one example of each where that is the most appropriate type to use.

CONTINUED

Peer Assessment

Explain your choices in Activity 8.2 to a partner. Did they come to the same conclusions as you did? Is there always a correct answer or are the different points of view all valid?

Boolean Operators

There are three **Boolean operators** that you can use to join conditions: the **AND operator**, the **NOT operator** and the **OR operator**. These are described in Table 8.4.

Boolean operator	Description	Example

AND	If both conditions are true, the result is true.
	
	If one or both conditions are false, the result is false.
		`IF 1 = 1 AND 2 = 2`
		
		This will return `TRUE`. The left of the `AND` is true, and the right of the `AND` is true.
		
		`IF 1 = 1 AND 1 > 2`
		
		This will return `FALSE`. The left of `AND` is true, but the right of `AND` is false.
		
		`IF 1 < -2 AND 0 < -1`
		
		This will return `FALSE`. Both comparisons are false, so the result is false.

OR	If one, or both, conditions are true, the result is true.
	
	If both conditions are false, the result is false.
		`IF 1 = 1 OR 2 = 2`
		
		This will return `TRUE`. The left of the `OR` is true, and the right of the `OR` is true.
		
		`IF 1 = 1 OR 1 > 2`
		
		This will return `TRUE`. The left of `OR` is true, but the right of `OR` is false.
		
		`IF 1 < -2 OR 0 < -1`
		
		This will return `FALSE`. Both comparisons are false, so the result is false.

NOT	Reverse the condition. If the condition is True it becomes False.	`IF NOT(1 = 1)`
		
		The brackets equal to `TRUE`, 1 equals 1. The `NOT` makes it `FALSE`, so it becomes 1 does not equal 1.
		
		`IF NOT (End of File)`
		
		This is used with file handing. `End of File` will return `TRUE` if there is no data left in the file. The `NOT` turns this to false. So while not at the end of the file.

Table 8.4: Boolean operators

KEY WORDS

Boolean operator: a symbol that joins multiple logical comparisons, can be `AND`, `OR` or `NOT`.

AND operator: returns True when both inputs are True.

NOT operator: returns True if the input is False, and False if it is True.

OR operator: returns True when one, or both, inputs are True.

ACTIVITY 8.3

Make a list of the use of AND, OR and NOT in real-life situations. For example, if one of two light switches is pressed then a light turns on. If the door is locked and you have the key then you can unlock the door.

Peer Assessment

Share your list in groups of 3. Discuss each of the statements and whether they have been correctly identified as AND, OR or NOT. Select one of each Boolean operator and share it with the rest of the class.

Example 1

This will output the first message if both test marks are greater than or equal to 90. If only one mark is greater than or equal to 90 then the second message will output.

```
OUTPUT("Enter the mark for test 1")
INPUT Mark1
OUTPUT("Enter the mark for test 2")
INPUT Mark2
IF Mark1 >= 90 AND Mark2 >= 90 THEN
   OUTPUT("Well done you got top marks on both tests")
ELSEIF Mark1 >= 90 OR Mark2 >= 90 THEN
   OUTPUT("Well done you got top marks on one of the tests")
ELSE
   OUTPUT("You didn't quite get top marks on the tests, try again next time")
ENDIF
```

VB.NET

```
Dim mark1 As Integer
Console.WriteLine("Enter the mark for test 1")
mark1 = Console.ReadLine
Dim mark2 As Integer
Console.WriteLine("Enter the mark for test 2")
mark2 = Console.ReadLine
If mark1 >= 90 And mark2 >= 90 Then
   Console.WriteLine("Well done you got top marks on both tests")
ElseIf mark1 >= 90 Or mark2 >= 90 Then
   Console.WriteLine("Well done you got top marks on one test")
Else
   Console.WriteLine("You didn't quite get top marks ont he tests, try again
next time")
End If
```

Python

```
mark1 = input("Enter the mark for test 1")
mark2 = input("Enter the mark for test 2")
if mark1 >= 90 and mark2 >= 90:
   print("Well done you got top marks on both tests")
elif mark1 >= 90 or mark2 >= 90:
   print("Well done you got top marks on one of the tests")
```

```
else:
    print("You didn't quite get top marks on the tests, try again next time")
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter the mark for test 1");
Integer mark1 = Integer.parseInt(scanner.nextLine());
    System.out.println("Enter the mark for test 2");
    Integer mark2 = Integer.parseInt(scanner.nextLine());
    if(mark1 >= 90 && mark2 >= 90){
        System.out.println("Well done you got top marks on both tests");
    }else if(mark1 >= 90 || mark2 >= 90){
        System.out.println("Well done you got top marks on one of the tests");
    }else{
        System.out.println("You didn't quite get top marks on the tests, try
            again next time");
    }
}
```

Example 2

Output the highest number out of three that are input:

```
OUTPUT("Enter 3 numbers")
INPUT Number1
INPUT Number2
INPUT Number3
IF Number1 > Number2 AND Number1 > Number3 THEN
    OUTPUT(Number1)
ELSEIF Number2 > Number3 THEN
    OUTPUT(Number2)
ELSE
    OUTPUT(Number3)
ENDIF
```

VB.NET

```
Dim number1 As Integer
Console.WriteLine("enter a number")
number1 = Console.ReadLine()
Dim number2 As Integer
Console.WriteLine("enter a number")
number2 = Console.ReadLine
Dim number3 As Integer
Console.WriteLine("enter a number")
number3 = Console.ReadLine
If number1 > number2 And number1 > number3 Then
    Console.WriteLine(number1)
ElseIf number2 > number3 Then
    Console.WriteLine(number2)
Else
    Console.WriteLine(number3)
End If
```

Python

```
number1 = input("Enter a number")
number2 = input("Enter a number")
number3 = input("Enter a number")
if number1 > number2 and number1 > number3:
    print(number1)
elif number2 > number3:
    print(number2)
else:
    print(number3)
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a number");
    Integer number1 = Integer.parseInt(scanner.nextLine());
    System.out.println("Enter a number");
    Integer number2 = Integer.parseInt(scanner.nextLine());
    System.out.println("Enter a number");
    Integer number3 = Integer.parseInt(scanner.nextLine());
    if(number1 > number2 && number1 > number3){
        System.out.println(number1);
    }else if(number2 > number3){
        System.out.println(number2);
    }else{
        System.out.println(number3);
    }
}
```

PROGRAMMING TASK 8.2

A computer program needs writing to act as a calculator. The program should take in two values and a symbol (e.g. +, −, * or /). Depending on the symbol entered, the calculator should perform that calculation. For example, if 3 5 + is entered, then the result should be 8 (3 + 5 = 8).

Getting started

1. Identify the inputs that the system will require.
2. Identify appropriate variables to store the inputs in.
3. Write a program to ask the user to enter the two numbers and symbol, and store these in variables.

Practice

1. Discuss in pairs which type of selection statement would be most appropriate for checking the symbol input.
2. Edit your program to use your chosen selection statement to check the symbol the user has entered. Depending on the symbol, perform the required calculation and output the result.

CONTINUED

Challenge

1 Discuss in pairs how the inputs could be repeatedly asked for until a valid entry is given. For example, keep entering a symbol until one of the valid ones is entered.

2 Implement your idea for repeatedly asking for the symbol to be input until a valid one is entered.

3 Include additional mathematical operations, for example, power of, modulus division.

Questions

8 Describe what is meant by selection.

9 Identify the **two** different examples of selection.

10 Write a program that takes two numbers as input and outputs the largest.

11 Write a program that outputs a question (e.g. a maths question), takes an answer from a user and outputs if they are correct or not.

12 Ask the user to input a colour. The program should then output a different message if the user enters the word `"yellow"`, `"green"` or `"blue"`. If neither of these are entered, the program should output a different message. Use a `CASE` statement.

8.7 Iteration

An **iteration** or **loop** is a programming construct where statements are run either a finite number of times, until a condition is true or while a condition is true.

There are three types of loop: **count-controlled**, **pre-condition** and **post-condition**.

Count-controlled

This type of loop uses a counter to run a set number of times. The most common count-controlled loop is the for loop. This has the structure:

```
FOR variable ← start value TO endvalue
    Code that runs repeatedly
NEXT variable
```

The loop will run from the start value to the end value, increasing by 1 each time. If the start value is 1 and the end value is 10, it will run 10 times (1, 2, 3, 4, 5, 6, 7, 8, 9 and 10).

Example 1

Output the numbers 1 to 10:

```
FOR X ← 1 TO 10
    OUTPUT(X)
NEXT X
```

KEY WORDS

iteration: a programming construct. Code is run multiple times – either a finite number of times (count-controlled), until a condition is true (post-condition), or while a condition is true (pre-condition).

loop: another name for iteration.

count-controlled loop: a type of iteration. Code is run a finite number of times. Usually a for loop.

pre-condition loop: a type of iteration. Code is run while the condition is true. The condition is checked before running any code in the loop, therefore the code might never run.

post-condition loop: a type of iteration. Code is run until a condition is true. The condition is checked after the code in the loop is run, therefore the code always runs once.

VB.NET

```
For x = 1 To 10
    Console.WriteLine(x)
Next
```

Python

```
for x in range(1, 11):
    print(x)
```

Java

```
public static void main(String args[]){
    for(Integer x = 1; x <= 10; x++){
        System.out.println(x);
    }
}
```

Example 2

Output the 12 times table from 1 to 12:

```
FOR Count ← 1 TO 12
    OUTPUT(Count * 12)
NEXT Count
```

VB.NET

```
For count = 1 To 12
    Console.WriteLine(count * 12)
Next
```

Python

```
for count in range (1, 13):
    print(count * 12)
```

Java

```
public static void main(String args[]){
    for(Integer count = 1; count < 13; count++){
        System.out.println(count * 12);
    }
}
```

Example 3

Add together the first 100 numbers:

```
Total ← 0
FOR Number ← 1 TO 100
    Total ← Total + Number
NEXT Number
```

VB.NET

```
Dim total As Integer = 0
For number = 1 To 100
    total = total + number
Next
```

Python

```
total = 0
for number in range(1, 101):
    total = total + number
```

Java

```
public static void main(String args[]){
    Integer total = 0;
    for(Integer number = 1; number <= 10; number++){
        total = total + number;
    }
}
```

You can change the amount that you increase the variable by each time you loop. This is by using `STEP`. `STEP 1` will increase the counter by 1 each time. `STEP -1` will decrease the counter by 1 each time. `STEP 0.5` will increase the counter by 0.5 each time.

Example 1

Output the numbers 10 to 1:

```
FOR Number ← 10 TO 1 STEP -1
    OUTPUT(Number)
NEXT Number
```

VB.NET

```
For number = 10 To 1 Step -1
    Console.WriteLine(number)
Next
```

Python

```
for number in range (10, 0, -1):
    print(str(number))
```

Java

```
public static void main(String args[]){
    for(Integer number = 10; number >= 1; number--){
        System.out.println(number);
    }
}
```

Example 2

Output the numbers from 11 to 20, increasing by 0.5 each time. (No example is given for Python as it does not support stepping in decimals.)

```
FOR Value ← 11 TO 20 STEP 0.5
    OUTPUT(Value)
NEXT Value
```

VB.NET

```
For value = 11 To 20 Step 0.5
    Console.WriteLine(value)
Next
```

Java

```
public static void main(String args[]){
    for(Double value = 11.0; value <= 20.0; value += 0.5){
        System.out.println(value);
    }
}
```

Pre-condition

A pre-condition loop tests the condition before starting the loop. This means that if the condition is false, the code inside the loop will not run. It loops while the condition is true. It stops looping when the condition is false.

A `WHILE` loop is a pre-condition loop. It has the structure:

```
WHILE condition DO
    Code that will run when the condition is true
ENDWHILE
```

Example 1

Looping while the user enters "Yes".

```
InputValue ← "Yes"
WHILE InputValue = "Yes" DO
    InputValue ← INPUT("Do you want to continue?")
ENDWHILE
```

VB.NET

```
Dim inputValue As String = "Yes"
While inputValue = "Yes"
    Console.WriteLine("Do you want to continue?")
    inputValue = Console.ReadLine
End While
```

Python

```
inputValue = "Yes"
while inputValue == "Yes":
    inputValue = input("Do you want to continue?")
```

Java

```
public static void main(String args[]){
    String inputValue = "Yes";
    Scanner scanner = new Scanner(System.in);
    while(inputValue.equals("Yes")){
        System.out.println("Do you want to continue?");
        inputValue = scanner.nextLine();
    }
}
```

Example 2

Outputting the numbers 1 to 10:

```
Number ← 1
WHILE Number < 11 DO
    OUTPUT(Number)
    Number ← Number + 1
ENDWHILE
```

VB.NET

```
Dim number As Integer = 1
While number < 11
    Console.WriteLine(number)
    number = number + 1
End While
```

Python

```
number = 1
while number < 11:
    print(str(number))
    number = number + 1
```

Java

```
public static void main(String args[]){
    Integer number = 1;
    while(number < 11){
        System.out.println(number);
        number++;
    }
}
```

Example 3

Asking the user to enter a number until they guess the stored number correctly:

```
Number ← 5
Guessed ← FALSE
WHILE Guessed = FALSE DO
    OUTPUT("Guess the number")
    INPUT Guess
    IF Guess = Number THEN
        Guessed ← TRUE
    ENDIF
ENDWHILE
```

VB.NET

```
Dim number As Integer = 5
Dim guessed As Boolean = False
While guessed = False
    Console.WriteLine("Guess the number")
    number = Console.ReadLine
    If guessed = number Then
        guessed = True
    End If
End While
```

Python

```
number = 5
guessed = False
while guessed == False:
    guess = int(input("Guess the number"))
    if guess == number:
        guessed = True
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    Integer number = 5;
    Boolean guessed = false;
    while(guessed == false){
        System.out.println("Guess the number");
        Integer guess = Integer.parseInt(scanner.nextLine());
        if(guess == number){
            guessed = true;
        }
    }
}
```

Post-condition loop

A post-condition loop runs the code inside the loop once, and then checks the condition at the end of the loop. This means that the code will always run once.

A `REPEAT UNTIL` loop is a post-condition loop. It has the structure:

```
REPEAT
    Code that runs inside the loop
UNTIL Condition
```

In this case it continues until the Condition becomes True. It loops while the condition is False.

Example 1

Looping until the user enters Yes. (No example is given for Python as it does not have an in-built post-condition loop.)

```
REPEAT
    OUTPUT("Do you want to stop?")
    INPUT Answer
UNTIL Answer = "Yes"
```

VB.NET

```
Dim answer As String
Do
    Console.WriteLine("Do you want to stop?")
    answer = Console.ReadLine
Loop Until answer = "Yes"
```

Java

Java has a do while loop, so it loops while the condition is true, not until it is true.

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    String answer = "Yes";
```

```
    do{
        System.out.println("Do you want to stop?");
        answer = scanner.nextLine();
    }while(!answer.equals("Yes"));
}
```

Example 2

Outputting the numbers 1 to 10. (No example is given for Python as it does not have an in-built post-condition loop.)

```
Number ← 1
REPEAT
    OUTPUT(Number)
    Number ← Number + 1
UNTIL Number > 10
```

VB.NET

```
Dim number As Integer = 1
Do
    Console.WriteLine(number)
    number = number + 1
Loop Until number > 10
```

Java

```
public static void main(String args[]){
    Integer number = 1;
    do{
        System.out.println(number);
        number++;
    }while(number <= 10);
}
```

Example 3

Asking the user to enter a number until they guess the correct number. (No example is given for Python as it does not have an in-built post-condition loop.)

```
NumberToGuess ← 15
REPEAT
    OUTPUT("Guess the number")
    INPUT Guess
UNTIL Guess = NumberToGuess
```

VB.NET

```
Dim numberToGuess As Integer = 15
Dim guess As Integer
```

```
Do
    Console.WriteLine("Guess the number")
    guess = Console.ReadLine
Loop Until guess = numberToGuess
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    Integer numberToGuess = 15;
    Integer guess;
    do{
        System.out.println("Guess the number");
        guess = Integer.parseInt(scanner.nextLine());
    }while(numberToGuess != guess);
}
```

PROGRAMMING TASK 8.3

A program needs to ask the user to guess what number the game is 'thinking of'. The game should store the number for the user to guess. The user should continually guess until they get the correct answer.

Getting started

1. Work in pairs to identify the inputs, processes and outputs required for this system.
2. Discuss which construct(s) will be needed: sequence, selection and/or iteration.
3. In pairs plan the algorithm to perform the required tasks.

Practice

1. Write a program for the algorithm you have designed.
2. Change the program so that the program outputs "lower" if their guess is too high, and "higher" if their guess is too low.

Challenge

1. Change the program to count how many times the user guesses the number before they get it correct. Output the total when they guess correctly.
2. Change the program to allow a user to enter the number for the player to guess at the start of the program.

SKILLS FOCUS 8.3

CONVERTING A FOR LOOP TO A WHILE LOOP

The three different types of loop (count-controlled, pre-condition and post-condition) can often be written as a different type of loop. For example, a count-controlled loop can be written using a pre-condition loop, or a post-condition loop. Pre-condition and post-condition loops can be rewritten as each other. Some pre- and post-condition loops can be written as a count-controlled - but only if their comparisons are for a count, e.g. looping 10 times.

A computational thinking skill is the ability to take a loop and convert it to other loops. This demonstrates your understanding of how the different loops work and the characteristics of each type of loop. Therefore it is good practice to experiment by converting one loop into a different type.

For example, converting a for loop to a while loop.

Consider the for loop:

```
FOR X ← 1 TO 10
    OUTPUT(X)
NEXT
```

Step 1: Declare the variable used as the counter. In this example the variable is x, the value is 1.

Step 2: Take the last value and put it in the while loop condition. In this example loop until it is 10, so the condition is while x < 11.

Step 3: Increment the counter in the loop. The counter is x so x needs to have 1 added to it.

```
X = 1   (Step 1)
WHILE X < 11 DO   (Step 2)
    OUTPUT(X)
    X ← X + 1   (Step 3)
ENDWHILE
```

Questions

1 Convert the following FOR loop to a WHILE loop.

```
FOR Count ← 0 TO 100
    OUTPUT(Count + Count)
NEXT
```

Step 1: Declare your variable with its starting value.

Step 2: Take the last value and put it in the while condition.

Step 3: Increment the counter in the loop.

2 Convert the following FOR loop to a WHILE loop.

```
FOR New ← 100 TO 111
    OUTPUT(New ^ New)
NEXT
```

Questions

13 Describe the difference between a pre-condition and post-condition loop.

14 A program needs a loop that will run 50 times. Which type of loop would be most appropriate?

15 Write a program to output the numbers 100 to 200.

16 Write a program to output the 5 times table (from 5 times 1, to 5 times 12).

17 Write a program to ask the user to enter a number continually, until they enter the number 10, using a post-condition loop.

18 Write a program to output the word "Hello" until the user enters the word "stop", using a pre-condition loop.

19 Convert the following count-controlled loop to a pre-condition loop.

```
FOR Counter ← 1 to 10
    OUTPUT(Counter * Counter)
NEXT Counter
```

8.8 Totalling

Totalling is adding together a set of values. To write a program to total you need to:

- Initialise the total to 0.
- Add the values together (either individually or within a loop).

KEY WORD

totalling: a type of program, it adds up multiple values to find the total.

Example 1

Asking the user to enter 10 numbers and totalling them:

```
Total ← 0
FOR Counter ← 0 TO 10
    OUTPUT("Enter a number")
    Total ← Total + INPUT
NEXT Counter
OUTPUT("The total is " & Total)
```

VB.NET

```
Dim total As Integer = 0
For counter = 0 To 10
    Console.WriteLine("Enter a number")
    total = total + Console.ReadLine
Next
Console.WriteLine("The total is " & total)
```

Python

```
total = 0
for counter in range(0, 11):
    total = total + int(input("Enter a number"))
print("The total is", total)
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    Integer total = 0;
    for(Integer counter = 0; counter < 11; counter++){
        System.out.println("Enter a number");
        total = total + Integer.parseInt(scanner.nextLine());
    }
    System.out.println("The total is " + total);
}
```

Example 2

Total the data in an array of 100 elements:

```
Total ← 0
FOR Count ← 0 TO 99
    Total ← Total + ArrayData[Count]
NEXT Count
OUTPUT(Total)
```

VB.NET

```
Dim total As Integer = 0
Dim arrayData(99) As Integer
'insert code to populate array
For count = 0 To 99
    total = total + arrayData(count)
Next
Console.WriteLine(total)
```

Python

```
total = 0
arrayData= []
#insert code to populate array
for count in range(0, 100):
    total = total + arrayData[count]
print(str(total))
```

Java

```
public static void main(String args[]){
    Integer total = 0;
    Integer[] arrayData = new Integer[100];
    //insert code to populate array
    for(Integer count = 0; count < 100; count++){
        total = total + arrayData[count];
    }
    System.out.println(total);
}
```

8.9 Counting

Counting is working out how many of something there are. For example how many numbers were entered that were over 10. To write a program to count you need to:

- Initialise a counter variable to 0.
- Increment (add one to) the counter each time an item is entered, or found.

KEY WORD

counting: a type of program, it adds one for every item to find out how many there are.

Example 1

Count how many numbers the user enters until they say to stop:

```
Count ← 0
Continue ← "Yes"
WHILE Continue = "Yes" DO
    OUTPUT("Do you want to continue?")
    INPUT Continue
    Count ← Count + 1
ENDWHILE
OUTPUT("You continued " & Count - 1 & " times")
```

VB.NET

```
Dim count As Integer = 0
Dim continueLoop As String = "Yes"
While continueLoop = "Yes"
    Console.WriteLine("Do you want to continue?")
    continueLoop = Console.ReadLine
    count = count + 1
End While
Console.WriteLine("You continued " & count-1 & " times")
```

Python

```
count = 0
continueInput = "Yes"
while continueInput == "Yes":
    continueInput = input("Do you want to continue?")
    count = count + 1
print("You continued", str(count-1), "times")
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter the first number");
    Integer count = 0;
    String continueInput = "Yes";
    while(continueInput.equals("Yes")){
        System.out.println("Do you want to continue?");
        continueInput = scanner.nextLine();
```

```
            count = count + 1;
        }
        count = count - 1;
        System.out.println("You continued " + count + " times");
    }
```

Example 2

Count how many numbers in an array of 100 elements are more than 50:

```
Count ← 0
FOR X ← 0 TO 99
    IF ArrayData[X] > 50 THEN
        Count ← Count + 1
    ENDIF
NEXT X
```

VB.NET

```
Dim arrayData(99) As Integer
'insert code to populate array

Dim count As Integer = 0
For X = 0 To 99
    If arrayData(X) > 50 Then
        count = count + 1
    End If
Next
```

Python

```
count = 0
arrayData=[]
#insert code to populate array
for x in range(0, 100):
    if arrayData[x] > 50:
        count = count + 1
```

Java

```
public static void main(String args[]){
    Integer[] arrayData = new Integer[100];
    //insert code to populate array
    Integer count = 0;
    for(Integer x = 0; x < 100; x++){
        if(arrayData[x] > 50){
            count = count + 1;
        }
    }
}
```

Questions

20 What are the **two** required elements for a totalling program.

21 What are the **two** required elements for a counting program.

22 Write a program to ask the user to input 100 numbers, total the values and output the total.

23 Write a program to ask the user to input numbers. Count how many numbers are less than 100, and how many are more than or equal to 100. Stop when the user enters the number 0.

8.10 String manipulation

A string is a piece of text. This could be made up of characters, numbers and/or symbols. There are lots of different string manipulators that you can use; these let you alter strings, find values in strings, etc. The two you need to know are length and substring.

Length

This command will return the number of characters in a string. It has the structure:

`LENGTH(string)`.

Example 1

`LENGTH("hi")` would return 2.

VB.NET

```
Dim stringLength As Integer
stringLength = Len("hi")
```

Python

```
stringLength = len("Hi")
```

Java

```
public static void main(String args[]){
    Integer stringLength = ("hi").length();
}
```

Example 2

`LENGTH("0123")` would return 4.

VB.NET

```
Dim stringLength As Integer
stringLength = Len("0123")
```

Python

```
stringLength = len("0123")
```

Java

```
public static void main(String args[]){
    Integer stringLength = ("0123").length();
}
```

Substring

This command will return some of the characters in the string. It has the structure:

`SUBSTRING(string, start character, number of characters).`

Depending on your language, the first character could be in position 0 or position 1.

Example 1

Using substring:

`SUBSTRING("Hello", 0, 1)` This will start at character `0` and take `1` character. It will return `"H"`.

VB.NET

(Uses 1 for the first character.)

```
Dim substring As String
substring = Mid("Hello", 1, 1)
```

Python

(Uses 0 for the first character.)

```
substring = "Hello"[0:1]
print(substring)
```

Java

```
public static void main(String args[]){
    String substring = ("Hello").substring(0,1);
    System.out.println(substring);
}
```

Example 2

Using substring:

`SUBSTRING("Goodbye", 4, 3)`. This will start at character `4` and take `3` characters. It will return `"bye"`.

VB.NET

```
Dim substring As String
substring = Mid("Goodbye", 5, 3)
```

Python

```
substring = "Goodbye"[4:7]
```

Java

```
public static void main(String args[]){
    String substring = ("Goodbye").substring(5, 3);
}
```

Example 3

Output the length of a string that the user inputs:

```
InputString ← INPUT("Enter a string")
StringLength ← LENGTH(InputString)
OUTPUT(InputString & " is " & StringLength & " characters long")
```

VB.NET

```
Dim inputString As String
Console.WriteLine("Enter a string")
inputString = Console.ReadLine
Dim stringlength As Integer
stringlength = Len(inputString)
Console.WriteLine(inputString & " is " & stringlength & " characters long")
```

Python

```
inputString = input("Enter a string")
stringLength = len(inputString)
print(inputString, " is ", str(stringLength), " characters long")
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a string");
    String inputString = scanner.nextLine();
    Integer stringLength = inputString.length();
    System.out.println(inputString + " is " + stringLength + " characters long");
}
```

Example 4

Output the first 4 characters in a string:

```
StringData ← "Goodbye"
NewMessage ← SUBSTRING(StringData, 0, 4)
OUTPUT(NewMessage)
```

VB.NET

```
Dim stringData As String = "Goodbye"
Dim newMessage As String
newMessage = Mid(stringData, 1, 4)
Console.WriteLine(newMessage)
```

Python

```
stringData = "Goodbye"
newMessage = stringData[0:4]
print(newMessage)
```

Java

```
public static void main(String args[]){
    String stringData = "Goodbye";
    String newMessage = stringData.substring(0,4);
    System.out.println(newMessage);
}
```

Example 5

Output each letter of a string one character at a time. Depending on your language, the stopping condition might be the length, or the length −1 depending on whether the first character is 0 or 1.

```
OUTPUT("Enter a message")
INPUT StringInput
FOR Count ← 0 to LENGTH(StringInput) - 1
    Character ← SUBSTRING(StringInput, Count, 1)
    OUTPUT(Character)
NEXT Count
```

VB.NET

```
Dim stringInput As String
Console.WriteLine("Enter a message")
stringInput = Console.ReadLine
Dim character As String
For count = 1 To Len(stringInput)
    character = Mid(stringInput, count, 1)
    Console.WriteLine(character)
Next
```

Python

```
stringInput = input("Enter a message")
for count in range(0, len(stringInput)):
    character = stringInput[count:count+1]
    print(character)
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a message");
    String stringInput = scanner.nextLine();
    String character;
    for(Integer count = 0; count < stringInput.length(); count++){
        character = stringInput.substring(count, count+1);
        System.out.println(character);
    }
}
```

Example 6

Output the last 3 characters in a string:

```
OUTPUT("Enter a message")
INPUT StringInput
NewString ← SUBSTRING(StringInput, LENGTH(StringInput) - 3, 3)
OUTPUT(NewString)
```

VB.NET

```
Dim stringInput As String
Console.WriteLine("Enter a message")
stringInput = Console.ReadLine
Dim newString As String
newString = Mid(stringInput, Len(stringInput) - 2, 3)
Console.WriteLine(newString)
```

Python

```
stringInput = input("Enter a message")
newString = stringInput[len(stringInput)-3:]
print(newString)
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a message");
    String stringInput = scanner.nextLine();
    String newString = stringInput.substring(stringInput.length()-3, stringInput.
length());
    System.out.println(newString);
}
```

Upper and lower

The characters a–z can be converted into uppercase and the characters A–Z can be converted into lowercase. This can be done to an individual character, or to an entire string at the same time. If a character is already in upper case, trying to convert it to upper case will not change it.

```
UPPER(string)
LOWER(string)
```

Example 1

Using `UPPER` with a string:

`UPPER("Hello")` will return `"HELLO"`

VB.NET

```
Word = "Hello".toUpper()
```

Python

```
Word = "Hello".upper()
```

Java

```
Word = "Hello".toUpperCase();
```

Example 2

Using `LOWER` with a string stored in a variable:

```
Word ← "HELLO"
Word ← LOWER(Word)
```

VB.NET

```
word = "HELLO"
word = word.toLower()
```

Python

```
word = "HELLO"
word = word.lower()
```

Java

```
word = "HELLO";
word = word.toLowerCase();
```

COMPUTER SCIENCE IN CONTEXT

When you need to create a password for a website or computer there are usually rules you have to follow; e.g. more than 8 characters, at least one lowercase letter, at least one uppercase letter, one special character, etc. The length function you have just learnt can be used to work out if the password is long enough. You can also use the substring function by checking each character one at a time to work out if it is a special character (e.g. / ! ?, etc.). You don't need to know about cases for the specification, but you can research how to find out about a character in upper case, or lowercase as well. Put them all together and you can write a program to check if a password is valid.

8.11 Nested statements

KEY WORD

nested statement: a construct (selection or iteration) that is inside another construct.

A **nested statement** is one or more selection and/or iteration statements inside another selection/iteration statement. This could be an IF statement inside an IF statement or a loop inside an IF statement or an IF statement in a loop or a loop within a loop. You might have already used these without realising they were called nested statements.

The position of the start and end of these constructs are important. If, for example, a loop starts inside an IF statement, the loop must also finish inside the same IF statement.

Example 1

Count how many numbers entered are more than 10, and how many are equal to 10:

```
MoreThan10 ← 0
EqualTo10 ← 0
FOR X ← 0 TO 99
    OUTPUT("Enter a number")
    INPUT Number
    IF Number > 10 THEN
        MoreThan10 ← MoreThan10 + 1
    ELSEIF Number = 10 THEN
        EqualTo10 ← EqualTo10 + 1
    ENDIF
NEXT X
```

This code has an `IF` statement nested inside a count-controlled loop.

VB.NET

```
Dim moreThan10 As Integer = 0
Dim equalTo10 As Integer = 0
Dim number As Integer
For x = 0 To 99
    Console.WriteLine("Enter a number")
    number = Console.ReadLine
    If number > 10 Then
        moreThan10 = moreThan10 + 1
    ElseIf number = 10 Then
```

```
        equalTo10 = equalTo10 + 1
    End If
Next
```

Python

```
moreThan10 = 0
equalTo10 = 0
for x in range(0, 100):
    number = int(input("Enter a number"))
    if number > 10:
        moreThan10 = moreThan10 + 1
    elif number = 10:
        equalTo10 = equalTo10 + 1
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    Integer moreThan10 = 0;
    Integer equalTo10 = 0;
    for(Integer x = 0; x < 100; x++){
        System.out.println("Enter a number");
        Integer number = Integer.parseInt(scanner.nextLine());
        if(number > 10){
            moreThan10 +=1;
        }else if(number == 10){
            equalTo10 +=1;
        }
    }
}
```

Example 2

Loop counting how many values in an array are more than or equal to 100, and then stop counting:

```
Number ← 0
Count ← 0
WHILE Number < 10 DO
    DataInArray ← ArrayData[Count]
    Count ← Count + 1
    IF DataInArray >= 100 THEN
        Number ← Number + 1
    ENDIF
ENDWHILE
```

This has an `IF` statement inside a pre-condition loop.

VB.NET

```
Dim number As Integer = 0
Dim count As Integer = 0
Dim dataArray(999) As Integer
'insert code to populate array

Dim dataInArray As Integer
While number < 10
    dataInArray = dataArray(count)
    count = count + 1
    If dataInArray >= 100 Then
        number = number + 1
    End If
End While
```

Python

```
arrayData = []
#insert code to populate array
number = 0
count = 0
while number < 10:
    dataInArray = arrayData[count]
    count = count + 1
    if dataInArray >= 100:
        number = number + 1
```

Java

```
public static void main(String args[]){
    Integer[] dataArray = new Integer[1000];
    //insert code to populate array
    Integer number = 0;
    Integer count = 0;
    Integer dataInArray = 0;
    while(number < 10){
        dataInArray = dataArray[count];
        count +=1;
        if(dataInArray >= 100){
            number +=1;
        }
    }
}
```

Example 3

Output only the vowels in a message input if user selects option 1:

```
OUTPUT("Enter 1 or 2")
INPUT Choice
IF Choice = 1 THEN
    OUTPUT("Enter a word")
    INPUT Word
    FOR Count ← 0 to LENGTH(Word)-1
        Character ← SUBSTRING(Word, Count, 1)
        IF Character = "a" OR Character = "e" OR Character = "I"
            OR Character = "o" OR Character = "u" THEN
            OUTPUT(Character)
        ENDIF
    NEXT Count
ENDIF
```

This has a `FOR` loop inside an `IF`, and another `IF` inside the `FOR` loop.

VB.NET

```
Dim choice As Integer
Console.WriteLine("Enter 1 or 2")
choice = Console.ReadLine
Dim word As String
Dim character As String
If choice = 1 Then
    Console.WriteLine("Enter a word")
    word = Console.ReadLine
    For count = 0 To Len(word)
        character = mid(word, count, 1)
        If character = "a" Or character = "e" Or character = "i" Or character =
            "o" Or character = "u" Then
            Console.WriteLine(character)
        End If
    Next
End If
```

Python

```
choice = int(input("Enter 1 or 2"))
if choice == 1:
    word = input("Enter a word")
    for count in range(0, len(word)):
        character = word[count:count+1]
        if character == "a" or character == "e" or character == "i" or character
            == "o" or character == "u":
            print(character)
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter 1 or 2");
    String word;
    String character;
    Integer choice = Integer.parseInt(scanner.nextLine());
    if(choice == 1){
        System.out.println("Enter a word");
        word = scanner.nextLine();
        for(Integer count = 0; count < word.length(); count++){
            character = word.substring(count, count + 1);
            if(character.equals("a") || character.equals("e") || character.
                equals("i") || character.equals("o") || character.equals("u")){
                System.out.println(character);
            }
        }
    }
}
```

Questions

24 Give **one** example of a nested statement.

25 What will the pseudocode statement `LENGTH("Hello World!")` return?

26 What will the pseudocode statement `SUBSTRING("HELLO WORLD!", 6, 5)` return?

27 Write a program to take a string input from the user, count out how many numbers are in the string and output the count.

28 Write a program to output a string value backwards.

8.12 Subroutines

A **subroutine** is a self-contained piece of code that has an identifier (name), and it can be called from anywhere in the main program.

When you decompose a problem into sub-systems, each of the sub-systems can be written as an individual subroutine. You can then call that subroutine when you need to use it.

Subroutines are useful because they reduce code. You write the subroutine once, and then you can call it as many times as you need to, instead of having to re-write it every time. Each time you re-write it there is a chance of an error, so this reduces the chances of this error.

There are two types of subroutine: **procedures** and **functions**. A function returns a value to the program that called it. A procedure does not return a value.

Procedures and functions can both take one or more values as **parameters**. These are values that are sent to the subroutine. Parameters will be introduced after the basics of procedures and functions.

KEY WORDS

subroutine: a self-contained piece of code that has an identifier and can be called from elsewhere in a program.

procedure: a subroutine that does not return a value to the program that called it.

function: a subroutine that does return a value to the program that called it.

parameter: a value that is sent to a subroutine.

Procedures

A procedure runs the code inside it, and does not return a value. The structure of a procedure is:

```
PROCEDURE identifier()
    code to run inside the function
ENDPROCEDURE
```

The identifier is then used in the main program.

Example 1

A procedure to output the numbers 1 to 10:

```
PROCEDURE Output1To10()
    FOR Count ← 1 to 10
        OUTPUT(Count)
    NEXT Count
ENDPROCEDURE
```

The main program can then call the procedure with the code:

```
Output1To10()
```

VB.NET

```
Sub Main()
    output1To10()
End Sub
Sub output1To10()
    For count = 1 To 10
        Console.WriteLine(count)
    Next
End Sub
```

Python

```
def output1To10():
    for count in range(1, 11):
        print(str(count))
output1To10()
```

Java

```
public static void output1To10(){
    for(Integer count = 0; count < 11; count++){
        System.out.println(count);
    }
}
public static void main(String args[]){
    output1To10();
}
```

Example 2

A procedure to take two numbers from the user and multiply then together:

```
PROCEDURE Multiply()
    OUTPUT("Enter a number")
    INPUT Num1
    OUTPUT("Enter a second number")
    INPUT Num2
    Total ← Num1 * Num2
ENDPROCEDURE
```

The procedure can be called in the main program with the code:

```
multiply()
```

VB.NET

```
Sub Main()
    multiply()
End Sub
Sub multiply()
    Dim num1 As Integer
    Console.WriteLine("Enter a number")
    num1 = Console.ReadLine
    Dim num2 As Integer
    Console.WriteLine("Enter a second number")
    num2 = Console.ReadLine
    Dim total As Integer
    total = num1 * num2
End Sub
```

Python

```
def multiply():
    num1 = int(input("Enter a number"))
    num2 = int(input("Enter a second number"))
    total = num1 * num2
multiply()
```

Java

```
public static void multiply(){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a number");
    Integer num1 = Integer.parseInt(scanner.nextLine());
    System.out.println("Enter a number");
    Integer num2 = Integer.parseInt(scanner.nextLine());
    Integer total = num1 * num2;
}
public static void main(String args[]){
    multiply();
}
```

Function

A function returns a value to the program that called it. This can be by either using the RETURN command, or saving a value to the function's identifier. Once a value is returned, the function stops running, so it cannot have any code after otherwise this will not run.

It has the structure:

```
FUNCTION identifier()
    Code to run in the function
    RETURN value
ENDFUNCTION
```

When the function is called it returns a value, so something needs to happen with this value. It could be output, e.g.

```
OUTPUT(function identifier)
```

or it could be saved in a variable, e.g.

```
variable identifier = function identifier
```

Example 1

Write a function to ask the user to enter two values, add them together and return the value:

```
FUNCTION Multiply()
    OUTPUT("Enter a number")
    INPUT Num1
    OUTPUT("Enter another number")
    INPUT Num2
    RETURN Num1 * Num2
ENDFUNCTION
```

To output the return value the main program can use:

```
OUTPUT(Multiply())
```

VB.NET

```
Sub Main()
    Console.WriteLine(multiply())
End Sub
Function multiply()
    Dim num1 As Integer
    Console.WriteLine("Enter a number")
    num1 = Console.ReadLine
    Dim num2 As Integer
    Console.WriteLine("Enter a second number")
    num2 = Console.ReadLine
    Return num1 * num2
End Function
```

Python

```
def multiply():
    num1 = int(input("Enter a number"))
    num2 = int(input("Enter another number"))
    return num1 * num2
print(str(multiply()))
```

Java

```
public static Integer multiply(){
    Scanner scanner = new Scanner(System.in);
vSystem.out.println("Enter a number");
    Integer num1 = Integer.parseInt(scanner.nextLine());
    System.out.println("Enter a second number");
    Integer num2 = Integer.parseInt(scanner.nextLine());
    return(num1 * num2);
}
public static void main(String args[]){
    System.out.println(multiply());
}
```

Example 2

Write a function to total all the values in an array with 50 elements and then return the total:

```
FUNCTION TotalValues()
    Total ← 0
    FOR X ← 0 TO 49
        Total ← Total + Array[X]
    NEXT X
    RETURN Total
ENDFUNCTION
```

To store the return value in a variable in the main program:

```
Total = TotalValues()
```

VB.NET

```
Sub Main()
    Dim total As Integer
    total = totalValues()
End Sub
Function totalValues()
    Dim arrayData(49) As Integer
    'insert code to populate array
    Dim total As Integer = 0
    For x = 0 To 49
        total = total + arrayData(x)
    Next
    Return total
End Function
```

Python

```
def totalValues():
    arrayData = []
    #insert code to populate array
    total = 0
    for x in range(0, 50):
        total = total + arrayData[x]
    return total

total = totalValues()
```

Java

```
public static Integer totalValues(){
    Integer[] arrayData = new Integer[50];
    //insert code to populate array
    Integer total = 0;
    for(Integer x = 0; x < 50; x++){
        total = total + arrayData[x];
    }
    return total;
}
public static void main(String args[]){
    Integer total = totalValues();
}
```

Scope

The **scope** of a variable is the areas within a program that it can be accessed. There are two scopes: **global** and **local**.

If you declare a variable (or constant, or array) as global then it means it can be accessed by any part of the program. That includes the main program and any subroutines. In most languages this means declaring it at the top of the program.

Example 1

Declaring a global variable, then outputting its value twice. Once in the main program, and once in a procedure call call.

```
GLOBAL Data
PROCEDURE OutputData()
    OUTPUT(Data)
ENDPROCEDURE
//main program
Data ← 1
OUTPUT(Data)
OutputData()
```

KEY WORDS

scope: the sections in the code where the variable, or constant, can be accessed.

global scope: the variable or constant can be accessed from any part of the program.

local scope: the variable or constant can only be accessed in the subroutine it is declared within.

VB.NET

```
Module Program
    Dim data As Integer
    Sub outputData()
        Console.WriteLine(data)
    End Sub
    Sub Main(args As String())
        data = 1
        Console.WriteLine(data)
        outputData()
    End Sub
End Module
```

Python

```
data = 1
def outputData():
    print(str(data))
print(str(data))
outputData()
```

Java

```
class outputting{
    static Integer data = 1;
    public static void outputData(){
        System.out.println(data);
    }
    public static void main(String args[]){
        System.out.println(data);
        outputData();
    }
}
```

If you declare a variable (or constant, or array) as local, then it can only be accessed in the part of the code where it is declared. If you declare it first in a subroutine, then it can only be accessed within that subroutine. If you declare it in the main program, it can only be accessed in the main program.

Example 2

Creating a local variable to the main program and outputting it twice. Once in the main program, and once from a subroutine where it is sent as a parameter.

```
PROCEDURE OutputData(DataParameter)
    OUTPUT(DataParameter)
ENDPROCEDURE
Data ← 1
OUTPUT(DataParameter)
OutputData(Data)
```

VB.NET

```
Module Program
    Sub outputData(dataParameter)
        Console.WriteLine(dataParameter)
    End Sub
    Sub Main(args As String())
        Dim data As Integer
        data = 1
        Console.WriteLine(data)
        outputData(data)
    End Sub
End Module
```

Python

```
def outputData(dataParameter):
    print(str(dataParameter))
#main
data = 1
print(str(data))
outputData(data)
```

Java

```
class outputting{
    public static void outputData(Integer dataParameter){
        System.out.println(dataParameter);
    }
    public static void main(String args[]){
        Integer data = 1;
        System.out.println(data);
        outputData(data);
    }
}
```

Best practice restricts the use of global variables, because their memory is taken for the whole of the program and nothing else can use that memory space. If you declare them locally then when that part of the program finishes the memory location is freed. Local variables are more tricky to program because you need to send them as parameters between functions and make sure you return them back if they have changed.

Parameters

A parameter is a value that is sent from the main program to the subroutine (procedure or function). Parameters are declared inside the brackets after the subroutines name, e.g.

```
PROCEDURE identifier(parameter1, parameter2 ...)
ENDPROCEDURE
```

or

```
FUNCTION identifier(parameter1, parameter2 ...)
ENDFUNCTION
```

If a subroutine is declared with parameters, then it must be called with the same number of parameters. For example:

```
PROCEDURE Total(Num1, Num2)
ENDPROCEDURE
```

This has two parameters. When the procedure is called it must have 2 numbers sent to it. This could be numbers, e.g.

```
Total(1,2)
```

or variables. e.g.

```
Total(Number1, Number2)
```

Example 1

A function takes two numbers, divides them and returns the result:

```
FUNCTION Division(First, Second)
   RETURN First / Second
ENDFUNCTION
```

The main program sends 10 and 2, then outputs the return value.

```
OUTPUT(Division(10, 2))
```

VB.NET

```
Sub Main()
   Console.WriteLine(division(10, 2))
End Sub
Function division(first, second)
   Return first / second
End Function
```

Python

```
def division(first, second):
   return first / second

print(str(division(10,2)))
```

Java

```
public static Double division(Double first, Double second)
{
   return (first / second);
}
public static void main(String args[]){
   System.out.println(division(10.0,2.0));
}
```

Example 2

A procedure takes 2 values and outputs all the numbers between the first number to the second:

```
PROCEDURE OutputNumbers(Num1, Num2)
    FOR Count ← Num1 TO Num2
        OUTPUT Count
    NEXT Count
ENDPROCEDURE
```

The main program taking two values from the user.

```
OUTPUT("Enter the smallest number")
INPUT FirstNumber
OUTPUT("Enter the largest number")
INPUT SecondNumber
OutputNumbers(FirstNumber, SecondNumber)
```

VB.NET

```
Sub Main()
    Console.WriteLine("Enter the smallest number")
    Dim firstNumber As Integer = Console.ReadLine
    Console.WriteLine("Enter the largest number")
    Dim secondNumber As Integer = Console.ReadLine
    outputNumbers(firstNumber, secondNumber)
End Sub
Sub outputNumbers(num1, num2)
    For count = num1 To num2
        Console.WriteLine(count)
    Next
End Sub
```

Python

```
def outputNumbers(num1, num2):
    for count in range(num1, num2+1):
        print(str(count))

firstNumber = int(input("Enter the smallest number"))
secondNumber = int(input("Enter the largest number"))
outputNumbers(firstNumber, secondNumber)
```

Java

```
public static void outputNumbers(Integer num1, Integer num2){
    for(Integer count = num1; count <= num2; count++){
        System.out.println(count);
    }
}
```

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter the smallest number");
    Integer firstNumber = Integer.parseInt(scanner.nextLine());
    System.out.println("Enter the largest number");
    Integer secondNumber =Integer.parseInt(scanner.nextLine());
    outputNumbers(firstNumber, secondNumber);
}
```

Questions

29 What is the difference between a function and a procedure?

30 Consider the following function:

```
FUNCTION Calculate(Num1, Num2)
    Num1 ← Num1 * 2
    Num2 ← Num2 + Num1
    RETURN(Num1 + Num2)
ENDFUNCTION
```

What will the following statement output?

```
OUTPUT(Calculate(1,2))
```

31 Write a program statement to call the following function with the parameter 100 and output the return value.

```
FUNCTION FindValue(Number)
    Number ← Number + INPUT
    RETURN Number
ENDFUNCTION
```

32 Write a procedure to take three numbers as parameters and output the largest.

33 Write a function that takes two strings as parameters. It takes the first 3 characters of each string and combines them, returning the resulting string.

8.13 Library routines

A program library is a set of subroutines that are pre-written and that can be called within a program.

In some programming languages the operators for `MOD` and `DIV` are library functions. In other programming languages they are just operators. For example, `2 MOD 4` is the same as `MOD(2, 4)`.

Two other **library routines** that you need to know are `ROUND` and `RANDOM`.

KEY WORD

library routine: a pre-written subroutine that can be called from within a program.

ROUND

This will take a real number (decimal) and limit how many numbers there are after the decimal point.

For example `ROUND(10.123, 1)` will take the number `10.123` and only leave 1 number after the decimal point, returning `10.1`.

`ROUND(4.8293, 2)` will return `4.82`.

As with functions, the values it returns need to be used. This could be done by outputting the return value, or saving it in a variable, e.g.

```
RoundedValue ← ROUND(77.293, 1)
```

VB.NET

```
Dim roundedValue As Single
roundedValue = Math.Round(77.293, 1)
```

Python

```
roundedValue = round(77.293,1)
```

Java

```
public static void main(String args[]){
   double value = Math.round(77.23 * 10.0) / 10.0;
}
```

RANDOM

This will generate a random number between two values that it takes as parameters. For example, `RANDOM(10, 20)` will return a number between 10 and 20.

ACTIVITY 8.4

Is there such a thing as a random number? Research how computers generate random numbers and work out if there is such a thing as a truly random number. Find out why randomness is important in programming and what the potential consequences are of having a system that does not generate random numbers.

`RANDOM(1, 4)` will return a number between 1 and 4.

As with functions, the values it returns and therefore need to be used. This could be by outputting the return value, or saving it in a variable, e.g.

```
randomNumber = RANDOM(1, 100)
```

VB.NET

```
Dim randomNumber As Integer
Dim rand As Random = New Random
randomNumber = rand.Next(1, 101)
```

Python

```
import random
randomNumber = random.randint(1, 100)
```

Java

```
public static void main(String args[]){
    Random rand = new Random();
    Integer randomNumber = rand.nextInt(1000) + 1;
}
```

8.14 Maintainable programs

When you write a program there are several things to take into consideration to make it a **maintainable program**. This is so that when you come back to it in a week, or a year, you can still understand what all of the code does. It might be you are writing a program that someone else needs to understand, so you need to make it understandable to someone who does not know what the program does.

KEY WORD

maintainable program: a program that has key features to help it be understood at a later date.

Meaningful identifiers

Variables, constants, subroutines and arrays all have identifiers (names). If you call a variable `X`, then there is no indication of what it is storing or what its purpose is. If instead, it is called `Total`, then you know that it is storing a total.

The identifiers for subroutines are usually descriptions of their function. For example, a procedure to output the numbers 1 to 10 could be called `Function1`, but then there is no indication of what it does. Instead, it could be called `Output1To10`.

Comments

A **comment** is a description of a line of code, or section of code. To write a comment you use a special character or characters, for example, `//`. This tells the program not to execute the text after this symbol.

You do not need to comment every line of code, for example, the statement `Count = 0` does not need commenting, it is clear that it is storing 0 in the variable count.

KEY WORD

comment: text within a program to describe its function, it is not executed when the program is run.

Example 1

The function of the `FOR` loop is written as a comment:

```
FOR Count ← 0 TO 9 //Output the first 10 elements in the array
    OUTPUT(Array[Count])
NEXT Count
```

VB.NET

```
For count = 0 To 9 'output the first 10 elements in the array
    Console.WriteLine(arrayData(count))
Next
```

Python

```
for count in range(0, 10): output the first 10 elements in the array
    print(arrayData[count])
```

Java

```
public static void main(String args[]){
    Integer[] arrayData = new Integer[10];
    //insert code to populate the array

    //output the first 10 elements in the array
    for(Integer count = 0; count < 11; count++){
        System.out.println(arrayData[count])
    }
}
```

Example 2

```
OUTPUT("Enter a number")
INPUT Num1
OUTPUT("Enter a number")
INPUT Num2
//find and output the largest number
IF Num1 > Num2 THEN
    OUTPUT(Num1)
ELSE
    OUTPUT(Num2)
ENDIF
```

VB.NET

```
Dim num1 As Integer
Dim num2 As Integer
Console.WriteLine("Enter a number")
num1 = Console.ReadLine
Console.WriteLine("Enter a number")
num2 = Console.ReadLine

'find and output the largest number
If num1 > num2 Then
    Console.WriteLine(num1)
Else
    Console.WriteLine(num2)
End If
```

Python

```
num1 = int(input("Enter a number"))
num2 = int(input("Enter a number"))
# find and output the largest number
if num1 > num2:
    print(str(num1))
else:
    print(str(num2))
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a number");
    Integer num1 = Integer.parseInt(scanner.nextLine());
    System.out.println("Enter a number");
    Integer num2 = Integer.parseInt(scanner.nextLine());
    if(num1 > num2){
        System.out.println(num1);
    }else{
        System.out.println(num2)
    }
}
```

Subroutines

Subroutines help to split the code down into sections, especially when one subroutine may need to be called multiple times. This means that if you need to make any changes then you only need to make them once. For more on subroutines, look back at section 8.12.

ACTIVITY 8.5

Open a computer program that you have written. Check its maintainability. Edit the program to improve the maintainability. Present your before and after program and explain how you improved its maintainability.

Questions

34 Explain how subroutines help make a program maintainable.

35 Describe **two** other ways of making a program maintainable.

36 Write a program statement to generate a random number between 1 and 5.

37 Identify the result from the statement `ROUND(3.142, 1)`.